

Part Γ: Text Classification with RNNs

Αλλαγές στον κώδικα του rnn.py

Για Το 3ο μέρος της εργασίας χρειάζεται η τροποποίηση και να η επανεκτέλεση του κώδικα του RNN.py 30+ φορές.

Αντί για την δημιουργία πολλών παραλλαγών του προγράμματος, ο κώδικας του RNN.py έγινε port στο παρόν notebook, το οποίο είναι ένα πλήρως αυτοματοποιημένο pipeline εκτέλεσης όλων των παραλλαγών των υποερωτημάτων.

Οι αλλαγές που έκανα περιλαμβάνουν τα εξής:

- Επέκταση της Αρχιτεκτονικής της κλάσης model. Στο notebook, η κλάση αντικαταστάθηκε (επεκτάθηκε) από την κλάση RNNClassifier που τώρα υποστηρίζει:
 - Εναλλαγή μεταξύ RNN και LSTM μέσω της παραμέτρου rnn_type,
 - Αμφίδρομο (Bidirectional) δίκτυα Μέσω της bidirectional=True, που αυτόματα διπλασιάζει το μέγεθος εισόδου στο Linear επίπεδο ταξινόμησης (hidden_dim * 2),
 - Πολλαπλά επίπεδα (Layers) Μέσω της num_layers (π.χ. 1 ή 2 επίπεδα).\

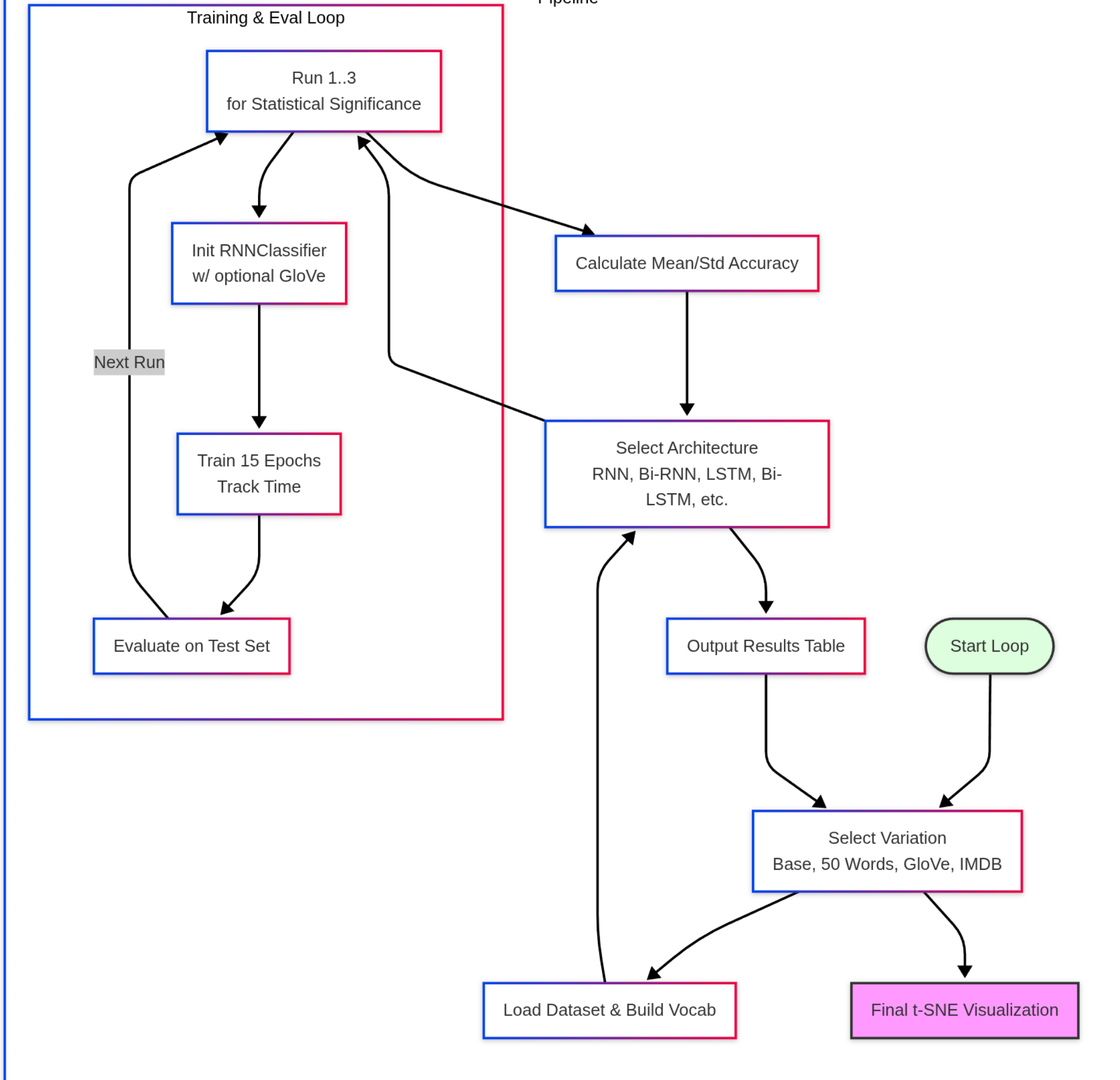
Με αυτόν τον τρόπο καλύφθηκαν οι 6 παραλλαγές του Ερωτήματος 1 (1RNN, 1Bi-RNN, 2Bi-RNN, 1LSTM, 1Bi-LSTM, 2Bi-LSTM).

- Υποστήριξη Pre-trained Embeddings (GloVe) Χρησιμοποιείται η βιβλιοθήκη gensim (μέσω του api.load('glove-wiki-gigaword-100')) για να κατεβούν και να φορτωθούν τα διανύσματα. Τα διανύσματα αυτά ανατίθενται απευθείας στο self.embedding_layer.weight.data. Επίσης, προστέθηκε η παράμετρος freeze_embeddings στην αρχιτεκτονική του μοντέλου, που θέτει το requires_grad = False στο layer των embeddings, εμποδίζοντας την ενημέρωση των βαρών τους κατά το training.
- Δυναμική Φόρτωση dataset & Υποστήριξη IMDB Η διαδικασία data loading μεταφέρθηκε μέσα σε μια συνάρτηση και προστέθηκε το flag use_imdb. Αν το flag είναι True, ο κώδικας φορτώνει το IMDB Dataset.csv, ορίζει ως labels το sentiment (θετικό/αρνητικό) και κάνει ένα δυναμικό split σε 80% train / 20% test, δημιουργώντας νέο δικό του λεξιλόγιο κατά την εκτέλεση.
- Αυτοματοποίηση Πειραματικού Βρόχου Για την αποφυγή της ανάγκης ανθρώπινης επίβλεψης κατά την εκπαίδευση και το evaluation όλων των παραλλαγών (μιας που το colab κάνει disconnect μετά από λίγη ώρα), δημιουργήθηκε μια κεντρική συνάρτηση run_variation(var_name, max_words, use_glove, freeze_embeddings, use_imdb) που για κάθε συνδυασμό υπερπαραμέτρων (max_words=25, max_words=50, GloVe trainable, GloVe frozen, IMDB), τρέχει αυτόματα και τις 6 αρχιτεκτονικές του Ερωτήματος 1.

Η Core λειτουργικότητα του rnn.py παραμένει, το port έγινε για διευκόλυνση της διαχείρισης του όγκου δοκιμών και παραλλαγών.

Παρατίθεται αρχιτεκτονικό διάγραμμα του execution loop του notebook:

Part_C_RNNs_Full_loop
Pipeline



Imports & Configuration

```
In [ ]: %pip install torch==2.3.0 torchttext==0.18.0
        %pip install pandas numpy tqdm scikit-learn matplotlib seaborn gensim
```

```
In [ ]: import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchttext.data import get_tokenizer
from torchttext.vocab import build_vocab_from_iterator
import pandas as pd
import numpy as np
import time
import copy
from tqdm import tqdm
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns
import gensim.downloader as api

sns.set_style('whitegrid')
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')
```

Hyperparameters

Toggle **MAX_WORDS** , **USE_GLOVE** , **FREEZE_EMBEDDINGS** , and **USE_IMDB** to run the variations.

```
In [2]: # Static Hyperparameters (These apply to all variations)
EPOCHS = 15
LEARNING_RATE = 1e-3
BATCH_SIZE = 1024
EMBEDDING_DIM = 100
HIDDEN_DIM = 64
NUM_RUNS = 3
```

Train/Eval Functions

```
In [3]: def train_model(model, loss_fn, optimizer, train_loader, epochs):
    epoch_times = []
    for epoch in range(1, epochs + 1):
        model.train()
        losses = []
        start = time.time()
        for X, Y in tqdm(train_loader, desc=f'Epoch {epoch}', leave=False):
            preds = model(X)
            loss = loss_fn(preds, Y)
            losses.append(loss.item())
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
        elapsed = time.time() - start
        epoch_times.append(elapsed)
```

```
print(f'Epoch {epoch:2d} | Loss: {np.mean(losses):.4f} | Time: {elapsed:.1f}s')
return epoch_times
```

```
In [4]: def evaluate_model(model, loss_fn, test_loader):
        model.eval()
        with torch.no_grad():
            Y_actual, Y_preds, losses = [], [], []
            for X, Y in test_loader:
                preds = model(X)
                loss = loss_fn(preds, Y)
                losses.append(loss.item())
                Y_actual.append(Y)
                Y_preds.append(preds.argmax(dim=-1))
            Y_actual = torch.cat(Y_actual)
            Y_preds = torch.cat(Y_preds)
        return (torch.tensor(losses).mean().item(),
                Y_actual.detach().cpu().numpy(),
                Y_preds.detach().cpu().numpy())
```

RNNClassifier

```
In [5]: class RNNClassifier(nn.Module):
        def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim,
                      rnn_type='RNN', bidirectional=False, num_layers=1,
                      pretrained_embeddings=None, freeze_embeddings=False):
            super(RNNClassifier, self).__init__()
            self.embedding_layer = nn.Embedding(num_embeddings=vocab_size, embedding_dim=embedding_dim)

            if pretrained_embeddings is not None:
                self.embedding_layer.weight.data.copy_(pretrained_embeddings)
                if freeze_embeddings:
                    self.embedding_layer.weight.requires_grad = False

            rnn_cls = nn.LSTM if rnn_type == 'LSTM' else nn.RNN
            self.rnn = rnn_cls(input_size=embedding_dim, hidden_size=hidden_dim,
                              num_layers=num_layers, batch_first=True,
                              bidirectional=bidirectional)

            linear_input_dim = hidden_dim * 2 if bidirectional else hidden_dim
            self.linear = nn.Linear(linear_input_dim, output_dim)
            self.rnn_type = rnn_type

        def forward(self, X_batch):
            embeddings = self.embedding_layer(X_batch)
            output, hidden = self.rnn(embeddings)
            logits = self.linear(output[:, -1])
            return F.softmax(logits, dim=1)
```

RNNClassifier Harness to run variations

```
In [6]: def run_variation(var_name, max_words, use_glove, freeze_embeddings, use_imdb):
        global global_glove

        print(f"\n{'#' * 80}")
        print(f"# RUNNING VARIATION: {var_name}")
        print(f"{'#' * 80}\n")
```

```

tokenizer = get_tokenizer('basic_english')

# Data Loading
if use_imdb:
    from torch.utils.data.dataset import random_split
    imdb_data = pd.read_csv('IMDB Dataset.csv')
    imdb_data['label'] = (imdb_data['sentiment'] == 'positive').astype(int)
    all_dataset = [(row['label'], row['review'].lower()) for _, row in imdb_data.iterrows()]
    train_size = int(0.8 * len(all_dataset))
    test_size = len(all_dataset) - train_size
    train_dataset, test_dataset = random_split(all_dataset, [train_size, test_size], generator=torch.Generator().manual_seed(42))
    train_dataset = list(train_dataset)
    test_dataset = list(test_dataset)
    target_classes = ['Negative', 'Positive']
else:
    train_data = pd.read_csv('ag-news-classification-dataset/train.csv')
    test_data = pd.read_csv('ag-news-classification-dataset/test.csv')
    train_dataset = [(label, train_data['Title'][i] + ' ' + train_data['Description'][i]) for i, label in enumerate(train_data['Class Index'])]
    test_dataset = [(label, test_data['Title'][i] + ' ' + test_data['Description'][i]) for i, label in enumerate(test_data['Class Index'])]
    target_classes = ['World', 'Sports', 'Business', 'Sci/Tech']

num_classes = len(target_classes)

# Vocab Building
def build_vocabulary(datasets):
    for dataset in datasets:
        for _, text in dataset:
            yield tokenizer(text)

vocab = build_vocab_from_iterator(build_vocabulary([train_dataset, test_dataset]), min_freq=10, specials=['<PAD>', '<UNK>'])
vocab.set_default_index(vocab['<UNK>'])

label_offset = 0 if use_imdb else 1

def collate_batch(batch):
    Y, X = list(zip(*batch))
    Y = torch.tensor(Y) - label_offset
    X = [vocab(tokenizer(text)) for text in X]
    X = [tokens + ([vocab['<PAD>']] * (max_words - len(tokens))) if len(tokens) < max_words else tokens[:max_words] for tokens in X]
    return torch.tensor(X, dtype=torch.int32).to(device), Y.to(device)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, collate_fn=collate_batch)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, collate_fn=collate_batch)

# GloVe Init
pretrained_vectors = None
if use_glove:
    print('Loading GloVe 6B-100d...')
    if global_glove is None:
        global_glove = api.load('glove-wiki-gigaword-100')
    pretrained_vectors = torch.zeros(len(vocab), EMBEDDING_DIM)
    found = 0
    for idx, word in enumerate(vocab.get_itos()):
        if word in global_glove:
            pretrained_vectors[idx] = torch.tensor(global_glove[word], dtype=torch.float32)
            found += 1
    print(f'GloVe vectors loaded. {found}/{len(vocab)} words found.')

all_results = {}
trained_models = {}

```

```

for arch in architectures:
    arch_name = arch['name']
    print(f"\n{'='*50}")
    print(f"  Architecture: {arch_name}")
    print(f"{'='*50}")
    run_accs, run_times = [], []
    n_params = None

    for run_idx in range(1, NUM_RUNS + 1):
        print(f"\n  --- Run {run_idx}/{NUM_RUNS} ---")
        model = RNNClassifier(
            vocab_size=len(vocab), embedding_dim=EMBEDDING_DIM, hidden_dim=HIDDEN_DIM,
            output_dim=num_classes, rnn_type=arch['rnn_type'], bidirectional=arch['bidirectional'],
            num_layers=arch['num_layers'], pretrained_embeddings=pretrained_vectors, freeze_embeddings=freeze_embeddings
        ).to(device)

        if n_params is None:
            n_params = count_parameters(model)
            print(f"  Parameters: {n_params:,}")

        loss_fn = nn.CrossEntropyLoss()
        optimizer = torch.optim.Adam([p for p in model.parameters() if p.requires_grad], lr=LEARNING_RATE)

        epoch_times = train_model(model, loss_fn, optimizer, train_loader, EPOCHS)
        _, Y_actual, Y_preds = evaluate_model(model, loss_fn, test_loader)
        acc = accuracy_score(Y_actual, Y_preds)

        run_accs.append(acc)
        run_times.append(np.mean(epoch_times))
        print(f'  Run {run_idx} Test Accuracy: {acc:.4f}')
        trained_models[arch_name] = model

    all_results[arch_name] = {
        'mean_acc': np.mean(run_accs), 'std_acc': np.std(run_accs),
        'params': n_params, 'mean_time_per_epoch': np.mean(run_times)
    }

print(f"\n{'Architecture':<15s} {'Mean Acc':>10s} {'Std Acc':>10s} {'Params':>12s} {'Time/Epoch':>12s}")
print('-' * 62)
for name, r in all_results.items():
    print(f"{name:<15s} {r['mean_acc']:>10.4f} {r['std_acc']:>10.4f} {r['params']:>12,} {r['mean_time_per_epoch']:>10.2f}s")

return all_results, trained_models, vocab

```

Setup and Main Loop

```

In [ ]: def count_parameters(model):
        return sum(p.numel() for p in model.parameters() if p.requires_grad)

architectures = [
    {'name': '1RNN',      'rnn_type': 'RNN',  'bidirectional': False, 'num_layers': 1},
    {'name': '1Bi-RNN',   'rnn_type': 'RNN',  'bidirectional': True,  'num_layers': 1},
    {'name': '2Bi-RNN',   'rnn_type': 'RNN',  'bidirectional': True,  'num_layers': 2},
    {'name': '1LSTM',     'rnn_type': 'LSTM',  'bidirectional': False, 'num_layers': 1},
    {'name': '1Bi-LSTM',  'rnn_type': 'LSTM',  'bidirectional': True,  'num_layers': 1},
    {'name': '2Bi-LSTM',  'rnn_type': 'LSTM',  'bidirectional': True,  'num_layers': 2},
]

```



```
# Load GloVe globally so it only downloads/parses once across all variations
global_glove = None
variations = [
    {"var_name": "Base (AG News, 25 words)", "max_words": 25, "use_glove": False, "freeze_embeddings": False, "use_imdb": False},
    {"var_name": "50 Max Words (AG News)", "max_words": 50, "use_glove": False, "freeze_embeddings": False, "use_imdb": False},
    {"var_name": "GloVe Trainable (AG News, 25 words)", "max_words": 25, "use_glove": True, "freeze_embeddings": False, "use_imdb": False},
    {"var_name": "GloVe Frozen (AG News, 25 words)", "max_words": 25, "use_glove": True, "freeze_embeddings": True, "use_imdb": False},
    {"var_name": "IMDB Dataset (25 words)", "max_words": 25, "use_glove": False, "freeze_embeddings": False, "use_imdb": True},
]

master_results = {}
base_rnn1_model = None
base_vocab = None

for var in variations:
    res, models, var_vocab = run_variation(**var)
    master_results[var['var_name']] = res

    # Save the 1RNN model and vocab from the Base variation for the t-SNE plot later
    if var['var_name'] == "Base (AG News, 25 words)":
        base_rnn1_model = models.get('1RNN')
        base_vocab = var_vocab

print('\n\nALL EXPERIMENTS COMPLETE!')
```

Τα outputs της πλήρους εκτέλεσης του loop έχουν αφαιρεθεί από το notebook λόγω μεγέθους, αλλά επισυνάπτονται παραδοτέο και βρίσκονται στο αρχείο `colab_outputs.txt`.

Ερώτημα 3.

t-SNE των Embeddings του 1RNN

```
In [ ]: tsne_words = [
    'business', 'career', 'student', 'university', 'college',
    'education', 'teacher', 'professor', 'school', 'degree',
    'economy', 'market', 'finance', 'investment', 'bank',
    'company', 'startup', 'entrepreneur', 'manager', 'salary',
    'science', 'research', 'technology', 'engineering', 'mathematics',
    'doctor', 'lawyer', 'accountant'
]

# Extract embeddings from the trained 1RNN model
rnn1_model = base_rnn1_model
if rnn1_model is None:
    print('ERROR: 1RNN model not found in trained_models dict.')
else:
    embedding_weights = rnn1_model.embedding_layer.weight.data.cpu().numpy()
    word_vectors = []
    valid_words = []
    for word in tsne_words:
        idx = base_vocab[word] # returns <UNK> index if not found
        if idx != base_vocab['<UNK>']:
            word_vectors.append(embedding_weights[idx])
            valid_words.append(word)
        else:
            print(f"'{word}' not in vocabulary, skipping.")

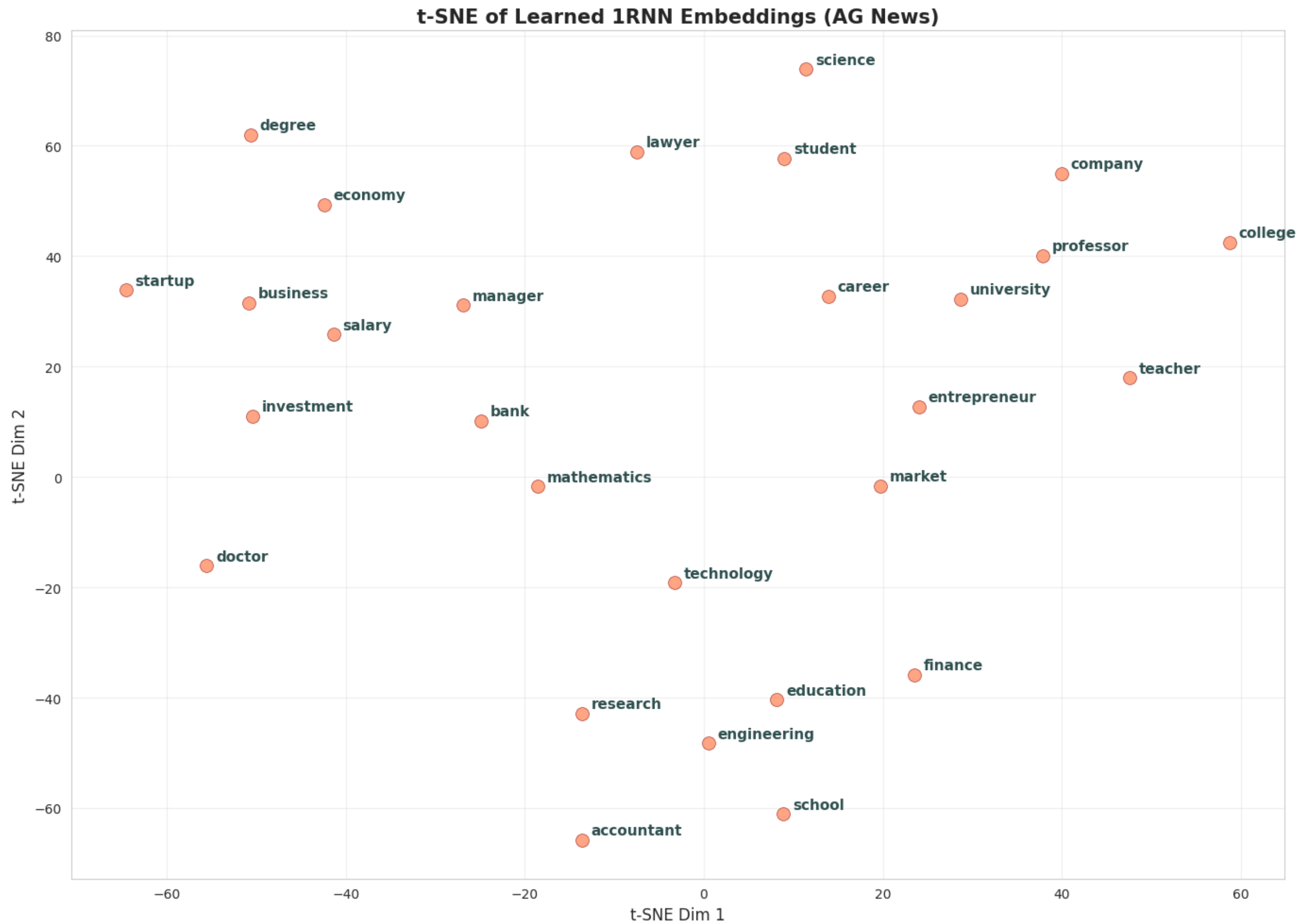
word_vectors = np.array(word_vectors)
print(f'Collected {len(valid_words)} word vectors')
```

```
# t-SNE
tsne = TSNE(n_components=2, random_state=42, perplexity=4, n_iter=2000)
embeddings_2d = tsne.fit_transform(word_vectors)

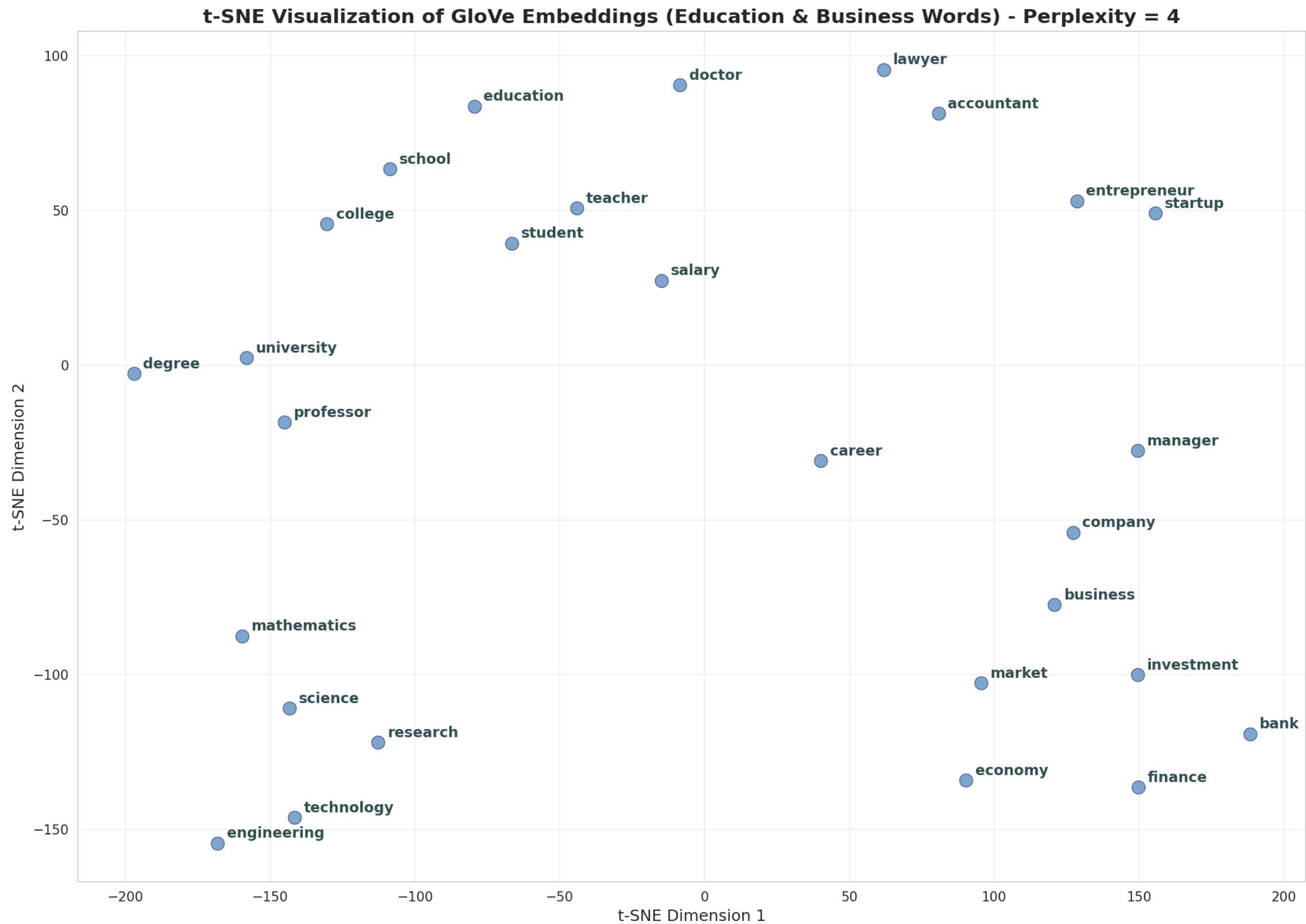
plt.figure(figsize=(14, 10))
plt.scatter(embeddings_2d[:, 0], embeddings_2d[:, 1],
            c='coral', s=100, alpha=0.7, edgecolors='darkred', linewidths=0.5)
for i, word in enumerate(valid_words):
    plt.annotate(word, xy=(embeddings_2d[i, 0], embeddings_2d[i, 1]),
                 xytext=(7, 4), textcoords='offset points',
                 fontsize=11, fontweight='bold', color='darkslategray')

plt.title('t-SNE of Learned 1RNN Embeddings (AG News)', fontsize=15, fontweight='bold')
plt.xlabel('t-SNE Dim 1', fontsize=12)
plt.ylabel('t-SNE Dim 2', fontsize=12)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('tsne_rnn_part_c.png', dpi=150, bbox_inches='tight')
plt.show()
print("Plot saved as 'tsne_rnn_part_c.png'")
```

Σύγκριση με ερώτημα A.6



Το γράφημα που βγάλαμε για το RNN με perplexity = 4,



Και το γράφημα του ερωτήματος Α.6, που το ξανατρέξαμε κι αυτό με perplexity = 4 για ομοιομορφία.

Μπορούμε να δούμε πολύ μεγάλη διαφορά στα 2 γραφήματα.

Αρχικά, του GloVe είναι πολύ πιο ξεκάθαρες οι συστάδες που σχηματίζονται, ενώ στου 1RNN είναι πιο διάχυτες οι λέξεις στον χώρο. από την άλλη, βλέπουμε και διαφορετικές έννοιες να σχηματίζουν ομάδες απο το ένα στο άλλο. Για παράδειγμα, το career φαίνεται να ομαδοποιείται με το university και το entrepreneurship στο 1rnn, ενώ στο gloVe είναι πολύ ξεκάθαρα εκτός συστάδας.Αντίστοιχα, το education ομαδοποιείται με τα engineering, research, finance ενώ στο glove είναι σε μια συστάδα πολύ πιο επικεντρωμένη γύρω από την εκπαίδευση.

Ερώτημα 1.

Εκτέλεση σε GPU (colab nVidia T4)

	1RNN	1Bi-RNN	2Bi-RNN	1LSTM	1Bi-LSTM	2Bi-LSTM
Mean Accuracy	0.8646	0.8664	0.8582	0.8799	0.8803	0.8858
Std. Accuracy	0.0051	0.0024	0.0100	0.0010	0.0004	0.0028
Parameters	2,136,284	2,147,164	2,171,996	2,168,156	2,210,908	2,310,236
Time cost	4.81s	4.92s	5.15s	5.08s	5.42s	5.76s

Εκτέλεση σε CPU (ryzen 5 4600H)

	1RNN	1Bi-RNN	2Bi-RNN	1LSTM	1Bi-LSTM	2Bi-LSTM
Mean Accuracy	0.8687	0.8675	0.8533	0.8816	0.8804	0.8864
Std. Accuracy	0.0064	0.0031	0.0056	0.0023	0.0010	0.0008
Parameters	2,136,284	2,147,164	2,171,996	2,168,156	2,210,908	2,310,236
Time cost	9.84s	15.18s	22.52s	14.60s	23.63s	43.20s

Σχολιασμός

Πώς επηρεάζεται η επίδοση του μοντέλου από την πολυπλοκότητά του?

Γενικά, η αύξηση της πολυπλοκότητας βελτιώνει την επίδοση (Mean Accuracy). Ειδικότερα, η μετάβαση από RNN (πιο απλό μοντέλο) σε LSTM (πιο περιπλοκο) την μεγαλύτερη βελτίωση, μιας που τα LSTM διαχειρίζονται πολύ καλύτερα το πρόβλημα των vanishing gradients και μαθαίνουν μακροπρόθεσμες εξαρτήσεις. Η χρήση αμφίδρομων (Bidirectional) μοντέλων αυξάνει επίσης την επίδοση, καθώς το μοντέλο λαμβάνει πληροφορία τόσο από τα προηγούμενα όσο και από τα επόμενα στοιχεία της ακολουθίας (κάτι που επιβεβαιώνεται και στον πίνακα: 1RNN: 0.8646 -> 1Bi-RNN: 0.8664 και 1LSTM: 0.8799 -> 1Bi-LSTM: 0.8803).

Πώς επηρρεάζεται το χρονικό κόστος από την πολυπλοκότητα του μοντέλου?

Οι χρονικές διαφορές είναι πολύ πιο έντοντς στην εκτέλεση στην CPU.

- Το 1LSTM χρειάζεται περίπου 50% περισσότερο χρόνο από το 1RNN, πράγμα λογικό λόγω των επιπλέον υπολογισμών στις πύλες.
- τα αμφίδρομα μοντέλα είναι πιο αργά από τα μονόδρομα, αφού επεξεργάζονται την ακολουθία 2 φορές
- Η προσθήκη 2ου στρώματος εκτοξεύει τον χρόνο (το 2Bi-LSTM είναι το πιο αργό με 5.76s), αφού αυξάνεται δραματικά ο αριθμός των παραμέτρων προς εκπαίδευση (από 2.1 εκατ. σε 2.3 εκατ.)

Κατά πόσο βοηθάει η χρήση 2 στρωμάτων στην βελτίωση της επίδοσης των μοντέλων?

- Στα LSTM, η χρήση 2 στρωμάτων βοηθάει (1Bi-LSTM: 0.8803 -> 2Bi-LSTM: 0.8858). Το πιο βαθύ δίκτυο καταφέρνει να εξάγει πιο σύνθετα χαρακτηριστικά και πετυχαίνει και την καλύτερη συνολική επίδοση στον πίνακα.

- Στα απλά RNN: Η χρήση 2 στρωμάτων παραδόξως μειώνει την επίδοση (1Bi-RNN: 0.8664 -> 2Bi-RNN: 0.8582). Αυτό πιθανώς οφείλεται στο πρόβλημα των vanishing gradientw. Κάνοντας ένα απλό RNN πιο "βαθύ", γίνεται πολύ πιο δύσκολο να εκπαιδευτεί σωστά, με αποτέλεσμα να έχει χειρότερη απόδοση παρότι έχει περισσότερες παραμέτρους.

Πόσο σταθερή είναι η επίδοση των μοντέλων σε κάθε επανάληψη?

Η σταθερότητα φαίνεται από την τυπική απόκλιση (Std. Accuracy στον πίνακα.)

Σε γενικές γραμμές, τα Bidirectional μοντέλα με 1 στρώμα (1Bi-RNN, 1Bi-LSTM) τείνουν να δίνουν τα πιο σταθερά αποτελέσματα (μικρότερο Std) σε σύγκριση με τα αντίστοιχα Unidirectional.

- Τα LSTM είναι εξαιρετικά σταθερά, με πολύ μικρές αποκλίσεις (1Bi-LSTM Std.Accuracy = 0.0004)
- Το 2Bi-RNN να είναι μακράν το πιο ασταθές μοντέλο (0.0100 απόκλιση στο run στην T4 GPU). Αυτό επιβεβαιώνει τη δυσκολία σύγκλισης κατά την εκπαίδευση ενός "βαθιού" απλού RNN.

Ερώτημα 2.

Στην πρώτη απόπειρα μετατροπής του κώδικα, επιχείρησα να χρησιμοποιήσω torch-directML για να εκμεταλλευτώ μια AMD rx480 (παλιά κάρτα που δεν υποστηρίζεται απο το RocM πια) που έχω και να τρέξω τον κώδικα τοπικά αντί του collab. Κατάφερε να τρέξει τον ported κώδικα κανονικά για τα RNN, αλλά όταν έφτασε στο 1LSTM έσκασε γιατί το torch-directML δεν υλοποιεί το aten::_thnn_fused_lstm_cell, που το ζητάει η υλοποίηση των LSTM στο torch.

Στην συνέχεια, έτρεξα το loop στον φορητό μου υπολογιστή και τέλος στο colab, με την free tier GPU (nvidia t4). Οι διαφορές είναι αναπάντεχες:

	1RNN	1LSTM
CPU (i5 4500)	~35sec	-
GPU (AMD rx480)	25.4sec	-
CPU (Ryzen 5 4600h)	9.84sec	14.60sec
GPU (nVidia T4)	4.81sec	5.08sec

Αρχικά, βλέπουμε πόση διαφορά κάνει η ηλικία του μηχανήματος. Η rx480 (του 2016), παρόλο που είναι gru, κάνει υπερδιπλάσιο χρόνο από ένα mobile class cpu (του 2021) και δεν σώζει ιδιαίτερο χρόνο από τον i5 4500, έναν πολύ παλιό πια επεξεργαστή.

Από την άλλη, η T4, ενώ είναι δυνατή enterprise grade κάρτα, κάνει μόνο τον μισό χρόνο για τα RNN και το 1/3 του χρόνου για τα LSTM σε σχέση με τον ryzen 5 4600h.

Σε σχέση με άλλες αρχιτεκτονικές Νευρωνικών Δικτύων, η μείωση του χρόνου με την χρήση επιταχυντή είναι πολύ μικρη και επιβεβαιώνεται, ειδικά αν λάβουμε υπόψιν τους χρόνους για τα 2-στρωματων RNN και LSTM, πως τα δίκτυα αυτά δεν παραλληλίζονται καλά.

Ερώτημα 4.

	1RNN	1Bi-RNN	2Bi-RNN	1LSTM	1Bi-LSTM	2Bi-LSTM
Mean Accuracy	0.4073	0.4800	0.4452	0.8816	0.8725	0.8897
Std. Accuracy	0.0207	0.0997	0.0522	0.0037	0.0170	0.0026
Parameters	2,136,284	2,147,164	2,171,996	2,168,156	2,210,908	2,310,236
Time cost	5.38s	5.72s	5.95s	5.76s	6.24s	6.39s

Με την αλλαγή του μήκους ακολουθίας φαίνεται το πραγματικό πρωτόρημα των LSTM και του μηχανισμού μνήμης τους.

Η αύξηση στις 50 λέξεις προκαλεί κατακόρυφη πτώση της επίδοσης στα απλά RNN (στο 1RNN πέφτει από το 0.8646 -> 0.4073 και η τυπική του απόκλιση εκτοξεύεται). Το ίδιο ισχύει για όλα τα RNN μοντέλα, αδυνατούν να "θυμηθούν" πληροφορίες από την αρχή μιας τόσο μεγάλης ακολουθίας και χάνουν τις μακροπρόθεσμες εξαρτήσεις στο κείμενο και αποτυγχάνουν να εκπαιδευτούν σωστά.

Αντίθετα, τα μοντέλα LSTM διατηρούν εξαιρετική επίδοση (γύρω στο 0.87 - 0.88), με το 2Bi-LSTM να παρουσιάζει μάλιστα μια μικρή βελτίωση (0.8897 έναντι 0.8858). Αυτό συμβαίνει επειδή τα LSTM διαθέτουν εσωτερικούς μηχανισμούς (gates) που ελέγχουν τη ροή της πληροφορίας, επιτρέποντάς τους να διατηρούν τη μνήμη τους σε μεγάλες ακολουθίες λέξεων.

Από πλευράς πολυπλοκότητας, Η παράμετρος max_words καθορίζει μόνο το μήκος της εισόδου στον χρόνο (time steps) και όχι το μέγεθος του λεξιλογίου, τη διάσταση των embeddings ή το μέγεθος των κρυφών επιπέδων Επομένως, ο αριθμός των παραμέτρων των μοντέλων μένει ίδιος.

Αξίζει να σημειωθεί επίσης ότι αυξάνεται αισθητά και ο χρόνος ανά εποχή.

Ερώτημα 5.

	1RNN	1Bi-RNN	2Bi-RNN	1LSTM	1Bi-LSTM	2Bi-LSTM
Mean Accuracy	0.8772	0.8756	0.8760	0.9085	0.9088	0.9082
Std. Accuracy	0.0167	0.0226	0.0113	0.0004	0.0002	0.0018
Parameters	2,136,284	2,147,164	2,171,996	2,168,156	2,210,908	2,310,236
Time cost	5.49s	5.24s	5.44s	5.32s	5.71s	6.01s

Η χρήση των =pre-trained word embeddings του GloVe επιφέρει σημαντικές βελτιώσεις στη συμπεριφορά των μοντέλων. Συγκρίνοντας αυτά τα αποτελέσματα με του ερωτήματος 1, μπορούμε να παρατηρήσουμε τα εξής:

- Βελτιώνεται θεαματικά η επίδοση σε όλα τα μοντέλα. Τα απλά RNN ανέβηκαν από το ~86% στο ~87.6%, ενώ τα LSTM έκαναν άλμα από το ~88% στο ~90.8%. Τα embeddings περιέχουν ήδη τις σημασιολογικές σχέσεις των λέξεων από ένα τεράστιο corpus, επομένως το μοντέλο δεν ξεκινάει από το μηδέν.
- Για τα LSTM, η χρήση του GloVe τα καθιστά ακόμα πιο σταθερά. Το 1Bi-LSTM έχει σχεδόν μηδενική απόκλιση (0.0002), πράγμα που δείχνει ότι συγκλίνει ιδανικά.
- Στα απλά RNN, η τυπική απόκλιση αυξήθηκε αισθητά (π.χ. στο 1Bi-RNN πήγε στο 0.0226). Αυτό το γεγονός πιθανόν να οφείλεται στην δυσκολία που έχουν τα απλά rnn να διαχειριστούν την πολύπλοκη και πυκνή πληροφορία στο input που προσφέρουν τα pretrained embeddings.
- Όπως και στο προηγούμενο ερώτημα, ο αριθμός παραμέτρων παραμένει ίδιος.
- Παρατηρείται μια μικρή αύξηση στο χρονικό κόστος, πολύ μικρότερη όμως από αυτή που παρατηρήσαμε στο ερώτημα 4.

Ερώτημα 6.

	1RNN	1Bi-RNN	2Bi-RNN	1LSTM	1Bi-LSTM	2Bi-LSTM
Mean Accuracy	0.8401	0.8604	0.8357	0.8979	0.8972	0.8974
Std. Accuracy	0.0039	0.0104	0.0344	0.0009	0.0007	0.0009
Parameters	10,884	21,764	46,596	42,756	85,508	184,836
Time cost	5.01s	5.12s	5.41s	5.30s	5.64s	6.05s

Σε σύγκριση με τα αποτελέσματα του προηγούμενου ερωτήματος, παρατηρούμε μερικές πολύ ενδιαφέρουσες και έντονες αλλαγές:

- Τεράστια πτώση στον αριθμό των παραμέτρων (π.χ. το 1RNN έπεσε από ~2.1 εκατομμύρια σε μόλις 10.884). Εφόσον τα embeddings έχουν γίνει "freeze", το δίκτυο πλέον εκπαιδεύει μόνο τα βάρη των κελιών (RNN/LSTM) και το τελικό γραμμικό επίπεδο (Linear layer) ταξινόμησης. Αυτό μειώνει τρομερά την υπολογιστική πολυπλοκότητα της εκπαίδευσης.

- Πτώση στην Επίδοση (Mean Accuracy) σε όλα τα μοντέλα: Τα απλά RNN έπεσαν από το ~87.6% στο ~84%-86% και τα LSTM έπεσαν από το ~90.8% στο ~89.7%. Μια πιθανή εξήγηση είναι πως αν και τα embeddings του GloVe προσφέρουν μια καλή γενική κατανόηση της γλώσσας, με το πάγωμα δεν επιτρέπουμε στο δίκτυο να τα προσαρμόσει (fine-tune) στο ύφος, το λεξιλόγιο και τις ιδιαιτερότητες του AG News dataset.
- Παρατηρείται μια μικρή αλλά αισθητή μείωση στον χρόνο ανά εποχή (π.χ. στο 1RNN από 5.49s σε 5.01s), καθώς χρειάζεται να κάνουμε backpropagation σε πολύ λιγότερα βαρη
- Μικρή αύξηση της σταθερότητας στα LSTM, μικρή μείωση στα απλά RNN, μιας που το βάρος της εκμάθησης πέφτει εξ ολοκλήρου στον ελάχιστο αριθμό παραμέτρων του RNN, κάνοντας την εκπαίδευση πιο δύσκολη και ευαίσθητη στην αρχικοποίηση.

Ερώτημα 7.

	1RNN	1Bi-RNN	2Bi-RNN	1LSTM	1Bi-LSTM	2Bi-LSTM
Mean Accuracy	0.7117	0.7095	0.7053	0.7153	0.7145	0.7179
Std. Accuracy	0.0041	0.0023	0.0018	0.0022	0.0013	0.0025
Parameters	2,917,254	2,928,006	2,952,838	2,949,126	2,991,750	3,091,078
Time cost	7.58s	7.80s	7.96s	7.84s	7.93s	8.15s

Σχολιασμός

Πώς επηρεάζεται η επίδοση του μοντέλου από την πολυπλοκότητά του?

Η αύξηση της πολυπλοκότητας (δηλαδή του αριθμού των παραμέτρων) δεν οδηγεί πάντα σε αναλογική βελτίωση της επίδοσης. Στα απλά RNN παρατηρούμε πως καθώς αυξάνεται η πολυπλοκότητα (από 1RNN σε 1Bi-RNN σε 2Bi-RNN), η μέση ακρίβεια αντί να αυξάνεται, μειώνεται (από 71.17% πέφτει στο 70.53%). Αυτό είναι ένδειξη ότι τα πιο πολύπλοκα απλά RNNs ίσως κανουν overfit στα δεδομένα. Όπως και πρίν, η μετάβαση στα LSTM βελτιώνει συνολικά την επίδοση. Εδώ, το πιο πολύπλοκο μοντέλο από όλα 2Bi-LSTM) καταφέρνει να σημειώσει και την υψηλότερη ακρίβεια.

Πώς επηρρεάζεται το χρονικό κόστος από την πολυπλοκότητα του μοντέλου?

Υπάρχει μια ξεκάθαρη συσχέτιση μεταξύ της πολυπλοκότητας του μοντέλου και της αύξησης του χρόνου εκπαίδευσης/εκτέλεσης. Καθώς αυξάνεται ο αριθμός των παραμέτρων (είτε προσθέτοντας bidirectionality, είτε 2ο στρώμα, είτε αλλάζοντας από RNN σε LSTM), ο χρόνος (Time cost) αυξάνεται αυστηρά. Αυτό το μέρος του πειράματος δεν το εκτέλεσα σε CPU, αλλά αναμένω πως θα ήταν μεγαλύτερη η διαφορά κατ'αναλογία του ερωτήματος 1.

Κατά πόσο βοηθάει η χρήση 2 στρωμάτων στην βελτίωση της επίδοσης των μοντέλων?

- Στο Bi-RNN, η προσθήκη δεύτερου στρώματος (μετάβαση από 1Bi-RNN σε 2Bi-RNN) μείωσε την ακρίβεια (από 70.95% σε 70.53%), πιθανότατα λόγω overfitting.
- Στο Bi-LSTM, η προσθήκη δεύτερου στρώματος (μετάβαση από 1Bi-LSTM σε 2Bi-LSTM) βελτίωσε την ακρίβεια (από 71.45% σε 71.79%). Συμπέρασμα: Το βάθος (2 στρώματα) βοηθά στην εξαγωγή πιο σύνθετων μοτίβων (features) κυρίως όταν το δίκτυο έχει ενσωματωμένους μηχανισμούς διαχείρισης της μνήμης.

Πόσο σταθερή είναι η επίδοση των μοντέλων σε κάθε επανάληψη?

Οι επιδόσεις των μοντέλων είναι πολύ σταθερές σε κάθε επανάληψη. Αξίζει μάλιστα να σημειωθεί ότι τα αμφίδρομα (bidirectional) μοντέλα τείνουν να είναι πιο σταθερά από τα αντίστοιχα απλά. Για παράδειγμα, το 1Bi-LSTM έχει την απόλυτα μικρότερη διακύμανση (0.0013).

Τελικές Ερωτήσεις

Υπάρχουν ερωτήματα που δεν καταφέρατε να απαντήσετε ή δεν είστε σίγουροι για την ορθότητα των απαντήσεών σας;

Στο ερώτημα 2 του τμήματος Γ, δεν κατάφερα να τρέξω όλο το πείραμα στην τοπική μου κάρτα γραφικών, όπως σχεδίαζα, και ότι κατάφερα να τρέξω ήταν με υποχωρήσεις. Π.χ, για να παίξει ο Adam

optimizer με το torch-directML χρειάστηκε να απενεργοποιήσω την παράμετρο foreach, η οποία batch-άρει element-wise operations και τρέχει πιο γρήγορα.

Ακόμα, δεν είμαι σίγουρος αν οι χρόνοι που απέδωσε η εν λόγω κάρτα (και που αναφέρω στο ερώτημα 2) είναι οι βέλτιστοι που θα μπορούσε ή αν υπήρχε καλύτερη προσέγγιση από την δική μου, που για compatibility layer χρησιμοποιεί τις βιβλιοθήκες του DirectX12. Αναρρωτιέμαι εαν η ίδια κάρτα θα απέδιδε καλύτερα σε περιβάλλον linux, παλιά έκδοση rocM και ενεργοποιημένο το foreach στον Adam, μην έχοντας τους περιορισμούς του directML.

Υπάρχει κάποιο πειραματικό αποτέλεσμα που σας φάνηκε απροσδόκητο;

Μου φάνηκαν απροσδόκητες κάποιες ομαδοποιήσεις στο t-sne του Μέρους A, όπως ότι το career απέχει πολύ απο την ακαδημαϊκή και την τεχνολογική συστάδα, ή ότι τα entrepreneur και startup δεν είναι μέσα στην συστάδα του business.

Επίσης μου έκανε εντύπωση η όχι-και-τόσο-μεγάλη βελτίωση χρόνου που έφερε η χρήση της T4 σε σχέση με τον επεξεργαστή του 5 ετών mid-range ultrabook μου! καταλαβαίνω πως σε μη-παραλληλίσιμες αρχιτεκτονικές μετράει περισσότερο το clock speed και το bandwidth, τελικά.